

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN - TIN HỌC

—o0o—



ĐỒ ÁN CUỐI KỲ
MÔN: PYTHON CHO KHOA HỌC DỮ LIỆU

ĐỀ TÀI: DỰ ĐOÁN GIÁ CƯỚC TAXI
SỬ DỤNG MÔ HÌNH MACHINE LEARNING

Họ tên sinh viên : Mai Quang Dũng - 232380049
Ngô Anh Khoa - 23280065

Lớp : 23KDL

Giảng viên : Hà Minh Tuấn

TP. Hồ Chí Minh, ngày 7 tháng 12 năm 2025

Mục lục

1	Giới thiệu	5
1.1	Bối cảnh và động lực	5
1.2	Mục tiêu đề án	5
1.3	Phạm vi dự án	5
2	Mô tả dữ liệu	6
2.1	Nguồn dữ liệu	6
2.2	Đặc trưng dữ liệu	6
2.2.1	Các biến số (Numeric Features)	6
2.2.2	Các biến phân loại (Categorical Features)	6
2.2.3	Biến mục tiêu (Target Variable)	6
2.3	Khám phá dữ liệu (EDA)	6
2.3.1	Tổng quan dữ liệu	7
2.3.2	Phân phối các biến số	7
2.3.3	Phân phối các biến phân loại	8
2.3.4	Ma trận tương quan	9
2.3.5	Phân tích biến mục tiêu	10
2.3.6	Phát hiện Outliers	10
3	Kiến trúc hệ thống và cấu trúc mã nguồn	11
3.1	Cấu trúc thư mục	11
3.2	Sơ đồ Pipeline	11
4	Tiền xử lý dữ liệu (Data Preprocessing)	12
4.1	Class DataLoader - Pre-split Processing	12
4.1.1	Mục đích và vai trò	12
4.1.2	Chi tiết các phương thức	13
4.1.3	Quy tắc ràng buộc dữ liệu (Constraint Rules)	13
4.2	Class DataTransformer - Post-split Processing	14
4.2.1	Nguyên tắc tránh Data Leakage	14
4.2.2	Chi tiết các phương thức	15
4.2.3	Các chiến lược xử lý Missing Values	15
4.2.4	Các phương pháp Encoding	16
4.2.5	Các phương pháp Scaling	16
5	Mô hình học máy (Machine Learning Models)	16
5.1	Kiến trúc Module Modeling	16
5.1.1	Class BaseTrainer - Lớp cơ sở trừu tượng	16
5.2	Các mô hình được sử dụng	17
5.2.1	Random Forest Regressor	17
5.2.2	Extra Trees Regressor	18
5.2.3	XGBoost Regressor	18
5.3	Class ModelTrainer - Orchestrator	19
5.3.1	Vai trò và chức năng	19
6	Tối ưu siêu tham số (Hyperparameter Optimization)	20
6.1	Giới thiệu về Hyperparameter Optimization	20

6.2	Framework Optuna	20
6.2.1	Các phương pháp tối ưu	21
6.2.2	Thuật toán TPE (Tree-structured Parzen Estimator)	21
6.2.3	Pruning - Dừng sớm các trials kém	21
6.2.4	Cấu hình Optuna trong dự án	21
6.3	Không gian tìm kiếm cho các mô hình	22
7	Kết quả thực nghiệm	22
7.1	Các chỉ số đánh giá (Evaluation Metrics)	22
7.1.1	RMSE - Root Mean Squared Error	22
7.1.2	MAE - Mean Absolute Error	23
7.1.3	R^2 - Coefficient of Determination	23
7.1.4	Tổng hợp ý nghĩa các Metrics	24
7.2	Kết quả so sánh các mô hình	25
7.2.1	Biểu đồ so sánh Metrics	25
7.2.2	Biểu đồ Actual vs Predicted	26
7.3	Phân tích chi tiết kết quả	26
7.3.1	Mô hình tốt nhất: XGBoost	26
7.3.2	Phân tích Overfitting	27
7.3.3	So sánh và nhận xét	27
7.4	Hyperparameters của các mô hình	28
7.5	Feature Importance	28
8	Trực quan hóa dữ liệu và kết quả	29
8.1	Class DataVisualizer	29
8.2	Danh sách các phương thức	29
9	Kết luận	29
9.1	Tổng kết	29
9.2	Hạn chế	30
9.3	Hướng phát triển	30
	Tài liệu tham khảo	30
	A Phụ lục: Danh sách thư viện sử dụng	31
	B Phụ lục: Phân công công việc	31

Danh sách hình vẽ

1	Tổng quan dữ liệu: thông tin về shape, dtypes và missing values	7
2	Phân phối của các biến số (numeric features) với histogram và KDE	7
3	Phân phối của các biến phân loại (categorical features)	8
4	Ma trận tương quan (Correlation Heatmap) giữa các biến số	9
5	Phân tích chi tiết biến mục tiêu Trip_Price	10
6	Boxplot phát hiện outliers cho tất cả biến số	10
7	Sơ đồ pipeline của hệ thống	11
8	So sánh các metrics (RMSE, MAE, R^2) của 3 mô hình	25
9	Scatter plot Actual vs Predicted cho 3 mô hình	26
10	So sánh Feature Importance của các tree-based models	28

Danh sách bảng

1	Mô tả các biến số trong dữ liệu	6
2	Mô tả các biến phân loại trong dữ liệu	6
3	Chi tiết các phương thức của lớp DataLoader	13
4	Quy tắc ràng buộc cho các biến số	14
5	Chi tiết các phương thức của lớp DataTransformer	15
6	So sánh các chiến lược xử lý Missing Values	15
7	So sánh các phương pháp Encoding	16
8	So sánh các phương pháp Scaling	16
9	Các phương thức của lớp BaseTrainer	17
10	Hyperparameters của Random Forest	18
11	Hyperparameters của Extra Trees	18
12	Hyperparameters của XGBoost	19
13	Các phương thức của lớp ModelTrainer	20
14	So sánh các phương pháp tối ưu hyperparameters	21
15	Không gian tìm kiếm hyperparameters	22
16	So sánh MAE và RMSE	23
17	Diễn giải giá trị R^2 trong các lĩnh vực	24
18	Tổng hợp các Evaluation Metrics	24
19	Kết quả đánh giá các mô hình trên tập test	25
20	Hyperparameters sử dụng cho từng mô hình	28
21	Chi tiết các phương thức trực quan hóa	29
22	Requirements.txt	31
23	Phân công công việc trong nhóm	31

1 Giới thiệu

1.1 Bối cảnh và động lực

Trong thời đại số hóa hiện nay, dịch vụ vận chuyển taxi đã trở thành một phần không thể thiếu trong cuộc sống hàng ngày. Việc dự đoán chính xác giá cước taxi không chỉ giúp hành khách có thể lập kế hoạch chi tiêu mà còn hỗ trợ các công ty vận tải trong việc tối ưu hóa dịch vụ.

Dự án này xây dựng một **pipeline học máy hoàn chỉnh** để dự đoán giá cước taxi dựa trên các yếu tố như:

- Khoảng cách di chuyển
- Thời gian di chuyển
- Số lượng hành khách
- Điều kiện giao thông
- Thời tiết
- Thời điểm trong ngày

1.2 Mục tiêu đồ án

Mục tiêu chính của đồ án bao gồm:

1. Xây dựng pipeline khoa học dữ liệu hoàn chỉnh từ tiền xử lý đến áp dụng mô hình vào thực tế
2. Áp dụng kỹ thuật lập trình hướng đối tượng (OOP) trong Python
3. So sánh hiệu suất của nhiều mô hình học máy
4. Tối ưu siêu tham số (hyperparameter) sử dụng Optuna
5. Trực quan hóa dữ liệu và kết quả

1.3 Phạm vi dự án

Dự án thực hiện bài toán **Regression** để dự đoán biến mục tiêu **Trip_Price** (giá cước taxi). Các mô hình được sử dụng bao gồm:

- Random Forest Regressor
- Extra Trees Regressor
- XGBoost Regressor

2 Mô tả dữ liệu

2.1 Nguồn dữ liệu

Dữ liệu được sử dụng là tập `taxi_price.csv` chứa thông tin về các chuyến đi taxi. Dữ liệu có thể được tải từ Google Drive thông qua script `main.py`.

2.2 Đặc trưng dữ liệu

2.2.1 Các biến số (Numeric Features)

Bảng 1: Mô tả các biến số trong dữ liệu

Tên biến	Kiểu	Mô tả
Trip_Distance_km	float	Khoảng cách di chuyển (km), giới hạn 0-140 km
Passenger_Count	int	Số lượng hành khách (1-6 người)
Base_Fare	float	Giá cước cơ bản
Per_Km_Rate	float	Giá mỗi km
Per_Minute_Rate	float	Giá mỗi phút
Trip_Duration_Minutes	float	Thời gian di chuyển (phút), giới hạn 0-120 phút
Speed_kmh	float	Tốc độ trung bình (km/h)

2.2.2 Các biến phân loại (Categorical Features)

Bảng 2: Mô tả các biến phân loại trong dữ liệu

Tên biến	Mô tả
Time_of_Day	Thời điểm trong ngày (Morning, Afternoon, Evening, Night)
Day_of_Week	Ngày trong tuần (Monday - Sunday)
Traffic_Conditions	Điều kiện giao thông (Low, Medium, High)
Weather	Điều kiện thời tiết (Clear, Rain, Fog, Snow)

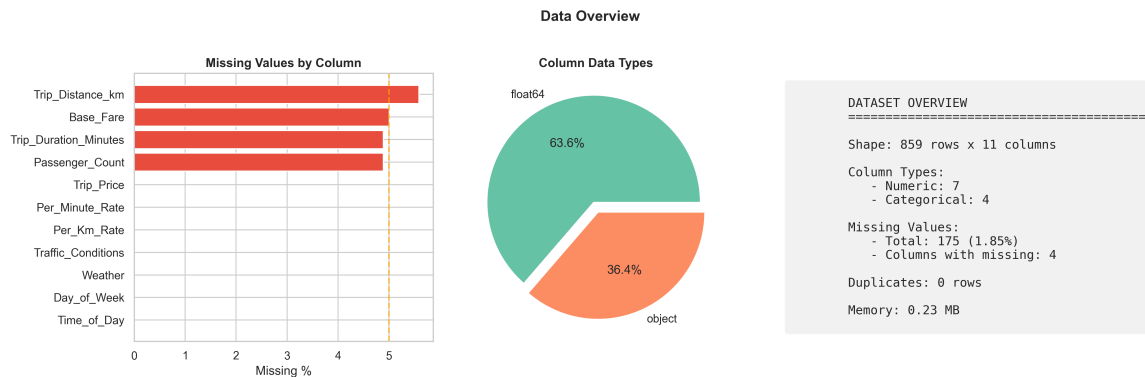
2.2.3 Biến mục tiêu (Target Variable)

- **Trip_Price**: Giá cước cuối cùng của chuyến đi (đơn vị tiền tệ)

2.3 Khám phá dữ liệu (EDA)

Trước khi xây dựng mô hình, chúng ta thực hiện Exploratory Data Analysis (EDA) để hiểu rõ dữ liệu. Các biểu đồ được tạo bởi class `DataVisualizer`.

2.3.1 Tổng quan dữ liệu

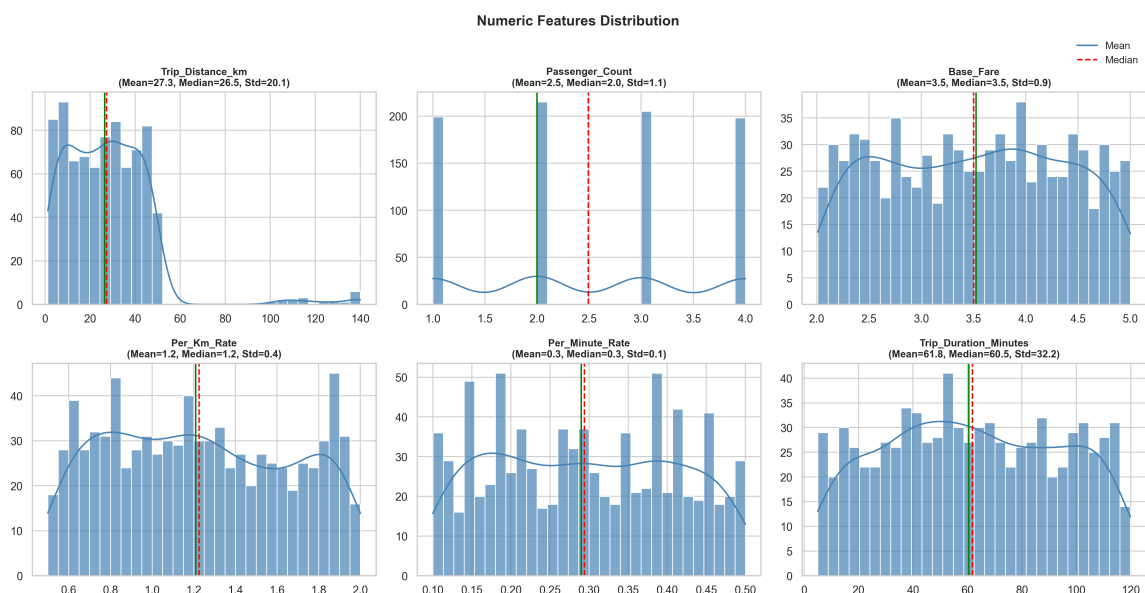


Hình 1: Tổng quan dữ liệu: thông tin về shape, dtypes và missing values

Hình 1 cho thấy:

- Dữ liệu có 1000 mẫu với 11 cột
- Không có missing values trong dữ liệu gốc
- 7 biến số (numeric) và 4 biến phân loại (categorical)

2.3.2 Phân phối các biến số

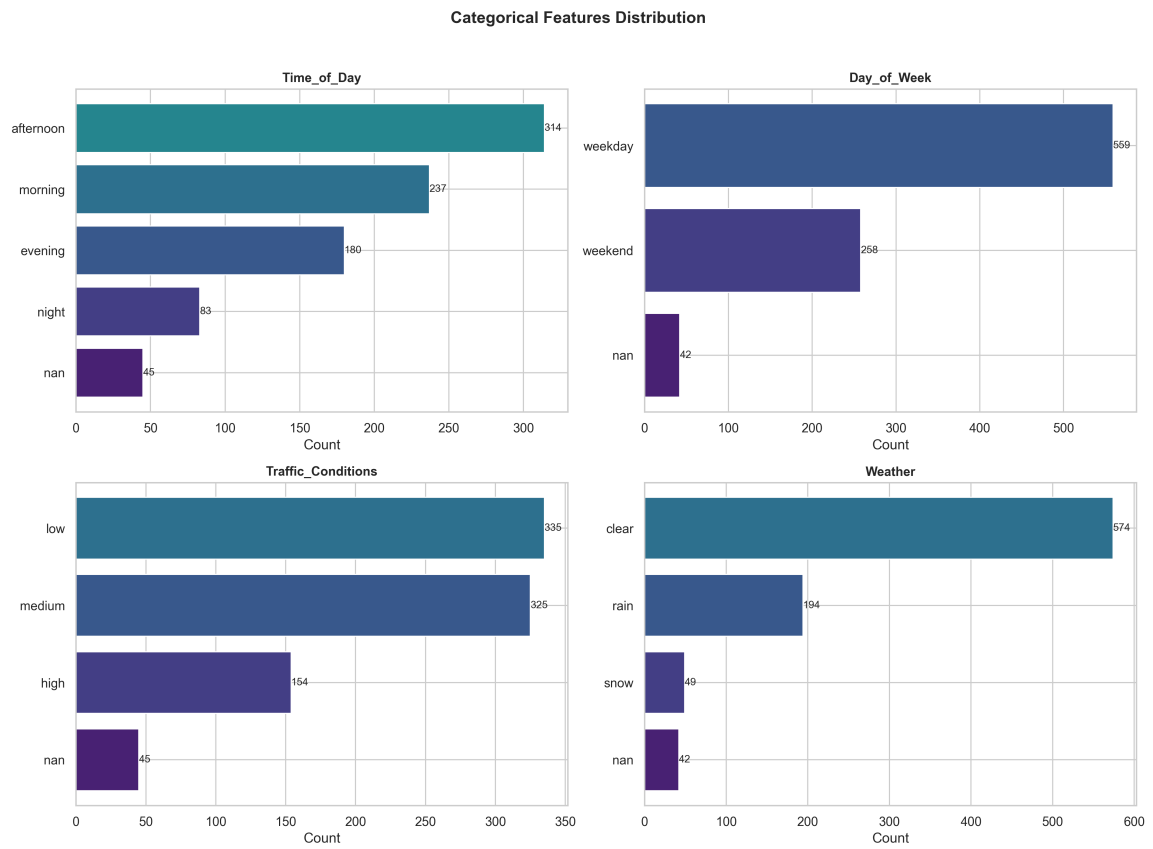


Hình 2: Phân phối của các biến số (numeric features) với histogram và KDE

Hình 2 hiển thị histogram kết hợp KDE cho các biến số, giúp nhận diện:

- Phân phối của từng biến (normal, skewed, multimodal)
- Các thống kê cơ bản: mean, median, standard deviation

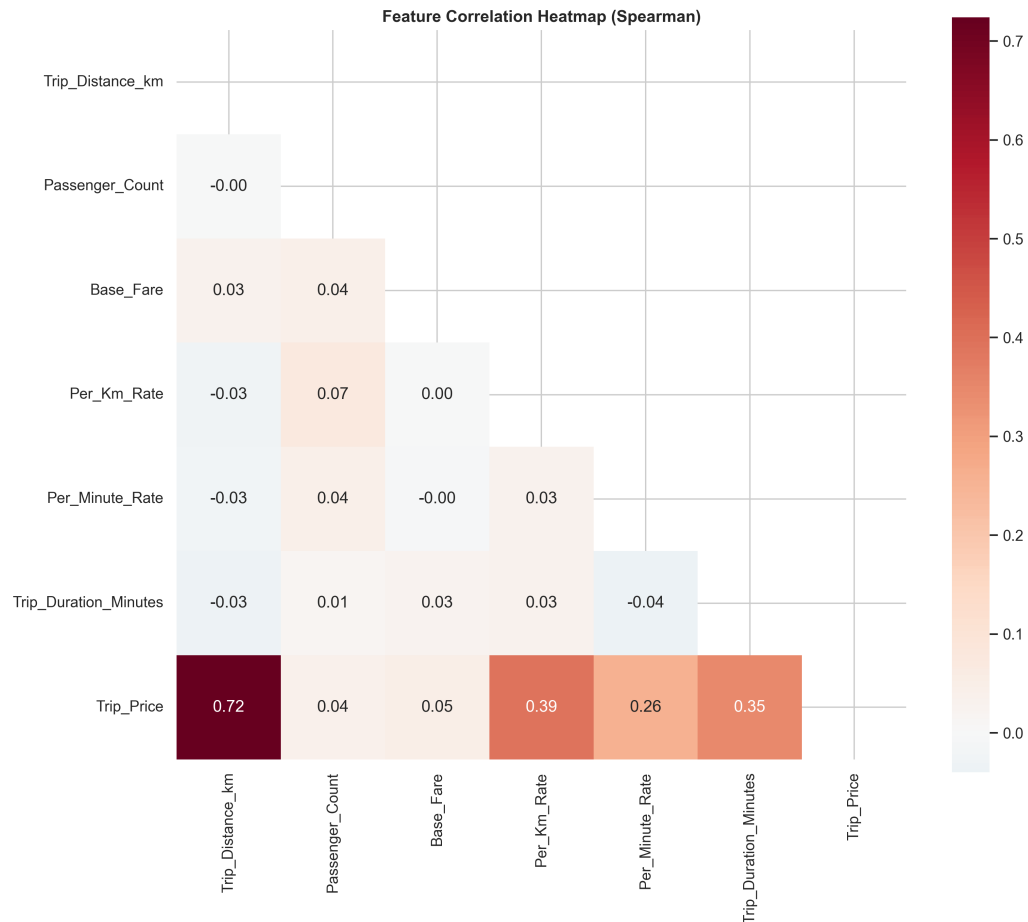
2.3.3 Phân phối các biến phân loại



Hình 3: Phân phối của các biến phân loại (categorical features)

Hình 3 cho thấy tần suất của từng category, giúp phát hiện class imbalance.

2.3.4 Ma trận tương quan



Hình 4: Ma trận tương quan (Correlation Heatmap) giữa các biến số

Cách đọc biểu đồ:

- Giá trị gần **1**: Tương quan dương mạnh (khi biến này tăng, biến kia cũng tăng)
- Giá trị gần **-1**: Tương quan âm mạnh (khi biến này tăng, biến kia giảm)
- Giá trị gần **0**: Không có tương quan tuyến tính

Nhận xét:

- **Trip_Distance_km** có tương quan cao nhất với **Trip_Price** → feature quan trọng
- Các features về giá (**Base_Fare**, **Per_Km_Rate**) cũng tương quan với target
- Cần kiểm tra multicollinearity giữa các features tương quan cao với nhau

2.3.5 Phân tích biến mục tiêu

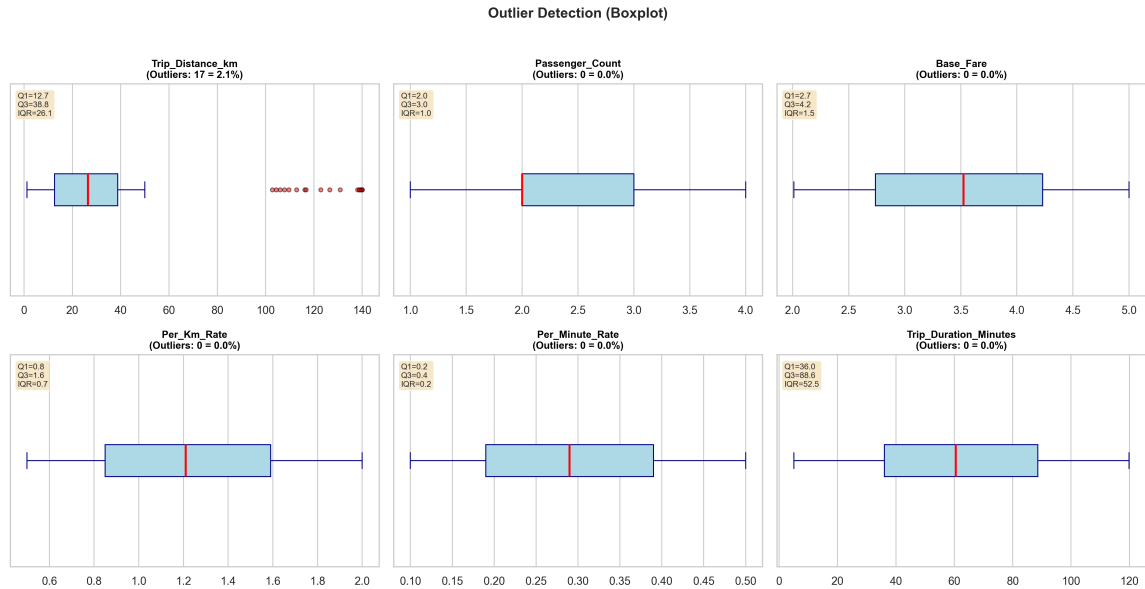


Hình 5: Phân tích chi tiết biến mục tiêu Trip_Price

Hình 5 phân tích biến mục tiêu Trip_Price:

- Distribution plot: Kiểm tra phân phối có gần normal không
- Q-Q plot: So sánh với phân phối chuẩn lý thuyết
- Box plot: Phát hiện outliers trong target

2.3.6 Phát hiện Outliers



Hình 6: Boxplot phát hiện outliers cho tất cả biến số

Hình 6 sử dụng boxplot để phát hiện outliers trong dữ liệu gốc. Các điểm nằm ngoài khoảng $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$ được coi là outliers và sẽ được xử lý trong bước tiền xử lý.

3 Kiến trúc hệ thống và cấu trúc mã nguồn

3.1 Cấu trúc thư mục

Dự án được tổ chức theo mô hình module hóa, giúp dễ dàng bảo trì và mở rộng:

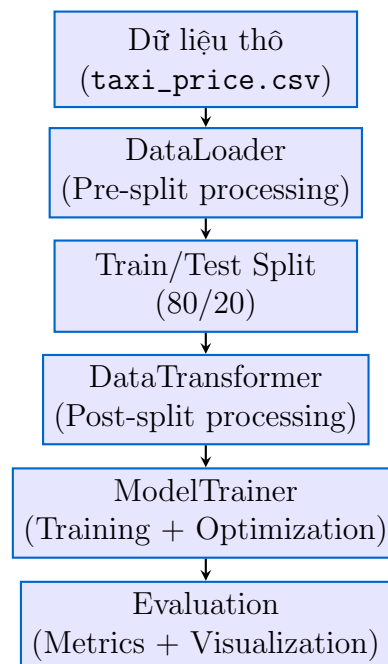
```

1 Do_An_Python/
2 |-- src/                                # Source code chính
3 |   |-- preprocessing/                  # Module tiền xử lý
4 |   |   |-- data_loader.py
5 |   |   |-- data_transformer.py
6 |   |-- modeling/                      # Module mô hình
7 |   |   |-- base_trainer.py
8 |   |   |-- model_registry.py
9 |   |   |-- model_trainer.py
10 |   |-- visualization/                 # Module trực quan hóa
11 |   |   |-- data_visualizer.py
12 |-- data/                             # Dữ liệu
13 |-- models/                           # Mô hình đã train
14 |-- results/                           # Kết quả và biểu đồ
15 |   |-- eda/                           # Biểu đồ EDA
16 |   |-- model/                         # Biểu đồ model
17 |-- notebooks/                         # Jupyter notebooks
18 |-- config.py                          # Cấu hình
19 |-- main.py                            # Script chính
20 |-- predict.py                         # Script dự đoán

```

Listing 1: Cấu trúc thư mục dự án

3.2 Sơ đồ Pipeline



Hình 7: Sơ đồ pipeline của hệ thống

4 Tiền xử lý dữ liệu (Data Preprocessing)

Module tiền xử lý được chia thành 2 phần chính để đảm bảo tránh data leakage:

4.1 Class `DataLoader` - Pre-split Processing

Lớp `DataLoader` xử lý các bước **trước** khi chia train/test, đảm bảo dữ liệu được làm sạch một cách nhất quán trước khi phân chia.

4.1.1 Mục đích và vai trò

`DataLoader` đóng vai trò là cổng vào đầu tiên của pipeline, chịu trách nhiệm:

- Đọc dữ liệu từ nhiều định dạng file khác nhau
- Phát hiện và phân loại tự động các kiểu dữ liệu
- Thực hiện các bước làm sạch cơ bản không phụ thuộc vào thống kê của tập train
- Cung cấp báo cáo EDA (Exploratory Data Analysis) tổng quan

4.1.2 Chi tiết các phương thức

Bảng 3: Chi tiết các phương thức của lớp DataLoader

Phương thức	Loại	Mô tả chi tiết
<code>load_data(filepath)</code>	Static method	Đọc dữ liệu từ file với các định dạng được hỗ trợ: CSV (.csv), Excel (.xls, .xlsx), JSON (.json). Tự động nhận diện định dạng dựa trên phần mở rộng file.
<code>from_file(filepath)</code>	Class method	Factory method tạo instance DataLoader mới từ file, tự động gọi <code>load_data()</code> và <code>detect_types()</code> .
<code>detect_types()</code>	Instance method	Phân loại tự động các cột thành 4 nhóm: numeric (số), categorical (phân loại), datetime (thời gian), boolean (logic). Sử dụng <code>pandas.select_dtypes()</code> .
<code>eda_overview(top_n)</code>	Instance method	Sinh báo cáo EDA bao gồm: shape, số duplicates, memory usage, tỉ lệ missing values, thống kê mô tả cho từng cột.
<code>drop_duplicates()</code>	Instance method	Loại bỏ các dòng trùng lặp hoàn toàn trong DataFrame, giữ lại dòng đầu tiên.
<code>unify_values(rules)</code>	Instance method	Chuẩn hóa giá trị text: chuyển lowercase, loại bỏ khoảng trắng thừa, thay thế các giá trị tương đương (ví dụ: "Yes"/"yes"/"Y" → "yes").
<code>apply_constraints</code>	Instance method	Áp dụng các ràng buộc miền giá trị: <i>clip</i> (cắt về min/max), <i>drop</i> (xóa dòng vi phạm), hoặc thay bằng mean/median.
<code>get_data()</code>	Instance method	Trả về DataFrame đã được xử lý, sẵn sàng cho bước chia train/test.

4.1.3 Quy tắc ràng buộc dữ liệu (Constraint Rules)

Hệ thống định nghĩa các quy tắc ràng buộc cho từng cột dữ liệu:

Bảng 4: Quy tắc ràng buộc cho các biến số

Biến	Min	Max	Action	Giải thích
Trip_Distance_km	0	140	clip	Giới hạn khoảng cách hợp lý
Passenger_Count	1	6	clip	Số hành khách tối thiểu 1, tối đa 6
Trip_Duration_Minutes	0	120	clip	Thời gian chuyến đi tối đa 2 giờ
Base_Fare	0	-	clip	Giá cước âm sẽ bị cắt về 0 (lỗi nhập liệu nhẹ)
Per_Km_Rate	0	-	drop	Giá/km âm bị xem là lỗi nặng → loại bỏ dòng
Per_Minute_Rate	0	-	drop	Giá/phút âm → drop để tránh nhiễu
Trip_Price	0	-	drop	Target không thể âm, loại bỏ để tránh hỏng training
Speed_kmh	0	160	clip	Vận tốc trung bình hợp lý (dùng khi bật feature)

4.2 Class DataTransformer - Post-split Processing

Lớp `DataTransformer` xử lý các bước **sau** khi chia train/test, đảm bảo nguyên tắc **fit trên train, transform trên test** để tránh data leakage.

4.2.1 Nguyên tắc tránh Data Leakage

Data Leakage là gì?

Data Leakage xảy ra khi thông tin từ tập test "rò rỉ" vào quá trình huấn luyện, dẫn đến đánh giá mô hình quá lạc quan. Ví dụ: nếu tính mean để fill missing trên toàn bộ dữ liệu (bao gồm test), mô hình sẽ "biết" thông tin từ tập test.

Để tránh data leakage, `DataTransformer`:

- Tính toán các thống kê (mean, median, mode) **chỉ từ tập train**
- Lưu lại các giá trị này để áp dụng cho tập test
- Fit các encoder/scaler trên train, chỉ transform trên test

4.2.2 Chi tiết các phương thức

Bảng 5: Chi tiết các phương thức của lớp DataTransformer

Phương thức	Tham số	Mô tả chi tiết
<code>fill_missing()</code>	rules, fit	Xử lý missing values: mean , median , mode , constant . Khi <code>fit=True</code> , tính và lưu giá trị thống kê.
<code>remove_outliers()</code>	method, threshold	Loại bỏ outliers: IQR ($1.5 \times \text{IQR}$), z-score (ngưỡng=3), Isolation Forest .
<code>encode_categorical</code>	method, cols	Mã hóa biến phân loại: OneHotEncoder , LabelEncoder , OrdinalEncoder .
<code>scale_features()</code>	method, exclude	Chuẩn hóa features số: StandardScaler , MinMaxScaler , RobustScaler .
<code>fit_transform()</code>	target_col	Pipeline hoàn chỉnh: fill missing → encode → scale. Tách target ra khỏi features.
<code>transform_new_data</code>	df	Áp dụng transformation đã học từ train lên dữ liệu mới.
<code>save_state()</code>	path	Lưu trạng thái (impute values, encoders, scalers) bằng joblib.
<code>load_state()</code>	path	Tải trạng thái đã lưu để transform dữ liệu mới.

4.2.3 Các chiến lược xử lý Missing Values

Bảng 6: So sánh các chiến lược xử lý Missing Values

Chiến lược	Ưu điểm	Nhược điểm	Phù hợp khi
Mean	Đơn giản, bảo toàn trung bình	Nhạy cảm với outliers, giảm variance	Dữ liệu phân phối đều
Median	Robust với outliers	Có thể không đại diện cho phân phối	Có outliers hoặc skewed
Mode	Phù hợp categorical	Có thể tạo bias nếu mode chiếm ưu thế	Biến phân loại
Constant	Kiểm soát hoàn toàn	Cần hiểu domain knowledge	Biết giá trị hợp lý

4.2.4 Các phương pháp Encoding

Bảng 7: So sánh các phương pháp Encoding

Phương pháp	Mô tả	Sử dụng khi
OneHotEncoder	Tạo k cột nhị phân cho k categories. VD: Weather có 4 giá trị $\rightarrow$ 4 cột mới.	Biến nominal (không có thứ tự), số categories ít.
LabelEncoder	Gán số nguyên 0, 1, 2... cho mỗi category.	Chỉ dùng cho target variable hoặc tree-based models.
OrdinalEncoder	Giống LabelEncoder nhưng cho phép định nghĩa thứ tự.	Biến ordinal có thứ tự (Low < Medium < High).

4.2.5 Các phương pháp Scaling

Bảng 8: So sánh các phương pháp Scaling

Phương pháp	Công thức	Ưu điểm	Nhược điểm
StandardScaler	$z = \frac{x-\mu}{\sigma}$	Chuẩn hóa về mean=0, std=1	Nhạy với outliers
MinMaxScaler	$x' = \frac{x-x_{min}}{x_{max}-x_{min}}$	Scale về [0,1], bảo toàn phân phối	Rất nhạy với outliers
RobustScaler	$x' = \frac{x-\text{median}}{IQR}$	Robust với outliers	Không scale về range cố định

5 Mô hình học máy (Machine Learning Models)

5.1 Kiến trúc Module Modeling

Module modeling được thiết kế theo pattern **Template Method** với abstract base class, cho phép dễ dàng mở rộng thêm các mô hình mới.

5.1.1 Class BaseTrainer - Lớp cơ sở trừu tượng

BaseTrainer là abstract base class định nghĩa interface chung cho tất cả các trainer:

Bảng 9: Các phương thức của lớp BaseTrainer

Phương thức	Loại	Mô tả
<code>__init__()</code>	Constructor	Nhận <code>X_train</code> , <code>X_test</code> , <code>y_train</code> , <code>y_test</code> và thiết lập các thuộc tính chung.
<code>model_name</code>	Property (abstract)	Trả về tên mô hình — phải được override bởi subclass.
<code>evaluate()</code>	Instance method	Tính các metrics đánh giá như RMSE, MAE, R^2 .
<code>_create_optuna_study()</code>	Instance method	Khởi tạo Optuna study dùng TPESampler và MedianPruner.
<code>_run_cross_validation()</code>	Instance method	Chạy k-fold cross-validation và trả về RMSE trung bình.
<code>_objective()</code>	Abstract method	Định nghĩa objective function cho Optuna.
<code>optimize()</code>	Abstract method	Tối ưu hyperparameters.
<code>train()</code>	Abstract method	Huấn luyện mô hình.

5.2 Các mô hình được sử dụng

5.2.1 Random Forest Regressor

Ý tưởng: Random Forest là thuật toán ensemble learning phổ biến, kết hợp nhiều Decision Trees được huấn luyện trên các bootstrap samples khác nhau (Bagging), sau đó lấy trung bình kết quả. Thuật toán xử lý tốt cả bài toán phân loại và hồi quy, ít nhạy cảm với outliers.

Công thức dự đoán:

$$\hat{y} = \frac{1}{B} \sum_{b=1}^B T_b(x) \quad (1)$$

Trong đó B là số lượng trees và $T_b(x)$ là dự đoán của tree thứ b .

Đặc điểm chính:

- **Bootstrap Aggregating (Bagging):** Mỗi tree được train trên một random sample (có hoàn lại) từ training data.
- **Random Feature Selection:** Tại mỗi node, chỉ xét một subset ngẫu nhiên của features.
- **Giảm Variance:** Kết hợp nhiều trees giúp giảm overfitting so với single tree.

Bảng 10: Hyperparameters của Random Forest

Tham số	Giá trị	Ý nghĩa
n_estimators	100	Số lượng trees trong forest. Nhiều hơn → ổn định hơn nhưng chậm hơn.
max_depth	10	Độ sâu tối đa của mỗi tree. Sâu hơn → phức tạp hơn → dễ overfit.
min_samples_split	5	Số mẫu tối thiểu để split một node. Lớn hơn → đơn giản hơn.
min_samples_leaf	2	Số mẫu tối thiểu ở mỗi leaf node.

5.2.2 Extra Trees Regressor

Ý tưởng: Extra Trees (Extremely Randomized Trees) là biến thể của Random Forest với tính ngẫu nhiên cao hơn. Tương tự Random Forest nhưng chọn split points ngẫu nhiên thay vì tối ưu, giúp giảm variance và huấn luyện nhanh hơn.

Khác biệt với Random Forest:

- **Split point:** Random Forest chọn split tối ưu trong subset features, Extra Trees chọn split *ngẫu nhiên*.
- **Bootstrap:** Random Forest dùng bootstrap samples, Extra Trees có thể dùng toàn bộ training data.
- **Bias-Variance:** Extra Trees có bias cao hơn một chút nhưng variance thấp hơn.

Bảng 11: Hyperparameters của Extra Trees

Tham số	Giá trị	Ý nghĩa
n_estimators	200	Số lượng trees (nhiều hơn RF do variance thấp hơn).
max_depth	12	Độ sâu tối đa (cho phép sâu hơn do random splits).
min_samples_split	2	Số mẫu tối thiểu để split (cho phép split sớm hơn).
min_samples_leaf	1	Số mẫu tối thiểu ở leaf (cho phép leaves nhỏ hơn).

5.2.3 XGBoost Regressor

Ý tưởng: XGBoost (Extreme Gradient Boosting) là thuật toán Gradient Boosting với các cải tiến về tối ưu hóa (second-order gradient), regularization mạnh, và xử lý missing values tự động. Khác với Random Forest (bagging), XGBoost huấn luyện các cây tuần tự, mỗi cây học từ sai sót của cây trước.

Công thức Boosting:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta \cdot f_t(x_i) \quad (2)$$

Trong đó η là learning rate và f_t là tree thứ t được train để sửa lỗi của các trees trước.

Objective Function của XGBoost:

$$\mathcal{L} = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{t=1}^T \Omega(f_t) \quad (3)$$

Với regularization term:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 + \alpha \sum_{j=1}^T |w_j| \quad (4)$$

Đặc điểm chính:

- **Second-order Gradient:** Sử dụng cả gradient và Hessian để tối ưu.
- **Regularization:** Kết hợp L1 (α) và L2 (λ) regularization.
- **Subsampling:** Cho phép subsample rows và columns để giảm overfitting.

Bảng 12: Hyperparameters của XGBoost

Tham số	Giá trị	Ý nghĩa
max_depth	4	Độ sâu tối đa (thường nhỏ hơn RF/ET).
learning_rate	0.05	Tốc độ học (shrinkage). Nhỏ hơn $\rightarrow$ cần nhiều trees hơn.
n_estimators	150	Số lượng boosting rounds.
subsample	0.7	Tỉ lệ samples sử dụng cho mỗi tree.
colsample_bytree	0.7	Tỉ lệ features sử dụng cho mỗi tree.
reg_lambda (λ)	1.0	L2 regularization trên leaf weights.
reg_alpha (α)	0.5	L1 regularization trên leaf weights.

5.3 Class ModelTrainer - Orchestrator

ModelTrainer là lớp điều phối (orchestrator) quản lý việc huấn luyện tất cả các mô hình.

5.3.1 Vai trò và chức năng

- **Quản lý trainers:** Tạo và quản lý các trainer cụ thể (RF, ET, XGBoost).
- **Đồng bộ kết quả:** Thu thập kết quả từ các trainers vào một nơi.
- **So sánh mô hình:** Tự động so sánh và chọn mô hình tốt nhất.
- **Lưu trữ:** Lưu tất cả mô hình và kết quả.

Bảng 13: Các phương thức của lớp ModelTrainer

Phương thức	Mô tả chi tiết
<code>train_rf()</code>	Huấn luyện Random Forest với các tham số <code>n_estimators</code> , <code>max_depth</code> , <code>min_samples_split</code> , <code>min_samples_leaf</code> .
<code>train_extra_trees()</code>	Huấn luyện Extra Trees với cấu hình tương tự Random Forest.
<code>train_xgb()</code>	Huấn luyện XGBoost với đầy đủ các hyperparameters của XGBoost.
<code>train_all(optimize)</code>	Huấn luyện tuần tự tất cả 3 mô hình. Nếu <code>optimize=True</code> , chạy Optuna optimization trước.
<code>optimize_*</code>	Các phương thức tối ưu hyperparameters cho từng mô hình bằng Optuna.
<code>compare_models()</code>	So sánh kết quả của tất cả mô hình, chọn và trả về mô hình có Test RMSE thấp nhất.
<code>save_all_models()</code>	Lưu tất cả mô hình đã train vào thư mục <code>models/</code> bằng <code>joblib</code> .
<code>save_results()</code>	Lưu kết quả đánh giá vào file JSON.

6 Tối ưu siêu tham số (Hyperparameter Optimization)

6.1 Giới thiệu về Hyperparameter Optimization

Hyperparameters là các tham số được thiết lập trước khi huấn luyện, không được học từ dữ liệu (khác với parameters như weights được học trong quá trình training).

Tại sao cần tối ưu hyperparameters?

- Hyperparameters ảnh hưởng lớn đến hiệu suất mô hình
- Không có bộ hyperparameters "tối ưu toàn cục phụ thuộc vào dữ liệu cụ thể"
- Tìm kiếm thủ công (manual tuning) tốn thời gian và không hiệu quả

6.2 Framework Optuna

Optuna là framework tối ưu hyperparameters hiện đại, được phát triển bởi Preferred Networks.

6.2.1 Các phương pháp tối ưu

Bảng 14: So sánh các phương pháp tối ưu hyperparameters

Phương pháp	Ưu điểm	Nhược điểm	Độ phức tạp
Grid Search	Đảm bảo tìm được tối ưu trong grid	Số lượng trials tăng theo hàm mũ	$O(n^k)$
Random Search	Hiệu quả hơn Grid Search	Không tận dụng kết quả trước	$O(n)$
Bayesian (TPE)	Tận dụng kết quả trước, hiệu quả cao	Phức tạp hơn	$O(n \log n)$

6.2.2 Thuật toán TPE (Tree-structured Parzen Estimator)

Optuna sử dụng thuật toán **TPE** để lấy mẫu thông minh:

1. **Chia trials thành 2 nhóm:** Nhóm tốt (top $\gamma\%$) và nhóm còn lại.
2. **Ước lượng phân phối:** Xây dựng $l(x)$ cho nhóm tốt và $g(x)$ cho nhóm còn lại.
3. **Maximize tỉ lệ:** Chọn hyperparameters tối đa hóa $\frac{l(x)}{g(x)}$.

Thực giác: TPE ưu tiên vùng không gian mà các trials trước đó cho kết quả tốt.

6.2.3 Pruning - Dừng sớm các trials kém

Optuna hỗ trợ **Pruning** để dừng sớm các trials không triển vọng:

- **MedianPruner:** Dừng trial nếu giá trị trung gian tệ hơn median của các trials trước.
- **Tiết kiệm tài nguyên:** Không cần chạy hết tất cả epochs/iterations cho trials tệ.

6.2.4 Cấu hình Optuna trong dự án

Dự án sử dụng Optuna với cấu hình sau:

- **Sampler:** TPESampler với random seed cố định (reproducibility)
- **Pruner:** MedianPruner
- **Direction:** Minimize (tối thiểu hóa RMSE)
- **Cross-validation:** 5-fold CV để đánh giá mỗi trial

6.3 Không gian tìm kiếm cho các mô hình

Bảng 15: Không gian tìm kiếm hyperparameters

Mô hình	Tham số	Kiểu	Range
Random Forest	n_estimators	int	[50, 300]
	max_depth	int	[5, 20]
	min_samples_split	int	[2, 10]
	min_samples_leaf	int	[1, 5]
Extra Trees	n_estimators	int	[50, 300]
	max_depth	int	[5, 25]
	min_samples_split	int	[2, 10]
	min_samples_leaf	int	[1, 5]
XGBoost	max_depth	int	[3, 10]
	learning_rate	float (log)	[0.01, 0.3]
	n_estimators	int	[50, 300]
	subsample	float	[0.5, 1.0]
	colsample_bytree	float	[0.5, 1.0]
	reg_lambda	float (log)	[0.01, 10]

7 Kết quả thực nghiệm

7.1 Các chỉ số đánh giá (Evaluation Metrics)

Để đánh giá hiệu suất của các mô hình hồi quy (regression), dự án sử dụng 3 chỉ số chính:

7.1.1 RMSE - Root Mean Squared Error

Định nghĩa RMSE

RMSE (Root Mean Squared Error) đo lường độ lệch trung bình giữa giá trị dự đoán và giá trị thực, với đơn vị giống biến mục tiêu.

Công thức:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (5)$$

Trong đó:

- n : Số lượng mẫu
- y_i : Giá trị thực của mẫu thứ i
- $\hat{y}_i$: Giá trị dự đoán của mẫu thứ i

Đặc điểm:

- **Đơn vị:** Cùng đơn vị với biến mục tiêu (VD: nếu Trip_Price tính bằng USD, RMSE cũng tính bằng USD)

- **Nhạy cảm với outliers:** Do bình phương sai số, các sai số lớn bị "phạt" nặng hơn
- **Luôn dương:** $RMSE \geq 0$, với $RMSE = 0$ là mô hình hoàn hảo
- **Ý nghĩa:** $RMSE = 6.77$ có nghĩa là trung bình, dự đoán lệch khoảng 6.77 đơn vị tiền tệ so với giá thực

Khi nào dùng RMSE?

- Khi muốn phạt nặng các sai số lớn
- Khi outliers quan trọng và cần được xem xét
- Là metric phổ biến nhất cho bài toán regression

7.1.2 MAE - Mean Absolute Error

Định nghĩa MAE

MAE (Mean Absolute Error) đo lường sai số trung bình tuyệt đối giữa dự đoán và thực tế.

Công thức:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (6)$$

Đặc điểm:

- **Đơn vị:** Cùng đơn vị với biến mục tiêu
- **Robust với outliers:** Không bình phương nên các sai số lớn không bị phạt quá mức
- **Dễ diễn giải:** $MAE = 4.36$ có nghĩa là trung bình, dự đoán lệch 4.36 đơn vị tiền tệ
- **Luôn $\leq$ RMSE:** Do bất đẳng thức Cauchy-Schwarz

So sánh MAE và RMSE:

Bảng 16: So sánh MAE và RMSE

Tiêu chí	MAE	RMSE
Nhạy cảm với outliers	Thấp	Cao
Gradient	Constant (± 1)	Tỉ lệ với sai số
Ý nghĩa trực quan	Sai số trung bình	Sai số "điển hình"
Khi $MAE \approx RMSE$	Sai số phân bố đều, ít outliers	
Khi $RMSE \gg MAE$	Có nhiều outliers hoặc sai số lớn	

7.1.3 R^2 - Coefficient of Determination

Định nghĩa R^2

R^2 (R-squared) đo lường tỉ lệ phương sai của biến mục tiêu được giải thích bởi mô hình.

Công thức:

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (7)$$

Trong đó:

- SS_{res} (Sum of Squared Residuals): Tổng bình phương sai số của mô hình
- SS_{tot} (Total Sum of Squares): Tổng bình phương độ lệch so với mean
- $\bar{y}$: Giá trị trung bình của biến mục tiêu

Đặc điểm:

- **Không có đơn vị:** R^2 là tỉ lệ, nằm trong khoảng $(-\infty, 1]$
- **Giá trị:**
 - $R^2 = 1$: Mô hình hoàn hảo
 - $R^2 = 0$: Mô hình không tốt hơn việc dự đoán bằng mean
 - $R^2 < 0$: Mô hình tệ hơn việc dự đoán bằng mean
- **Ý nghĩa:** $R^2 = 0.959$ có nghĩa mô hình giải thích được 95.9% phương sai của giá cước taxi

Diễn giải R^2 theo ngành:

Bảng 17: Diễn giải giá trị R^2 trong các lĩnh vực

Giá trị R^2	Diễn giải
$R^2 > 0.9$	Rất tốt (phổ biến trong physical sciences)
$0.7 < R^2 < 0.9$	Tốt (phổ biến trong social sciences)
$0.5 < R^2 < 0.7$	Trung bình
$R^2 < 0.5$	Yếu

7.1.4 Tổng hợp ý nghĩa các Metrics

Bảng 18: Tổng hợp các Evaluation Metrics

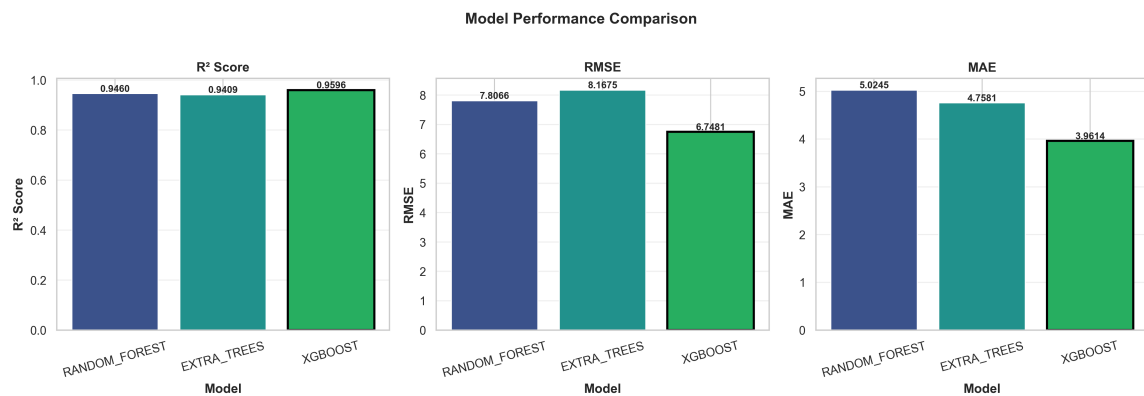
Metric	Range	Optimal	Đơn vị	Câu hỏi trả lời
RMSE	$[0, +\infty)$	0	Như target	"Sai số điển hình là bao nhiêu?"
MAE	$[0, +\infty)$	0	Như target	"Sai số trung bình là bao nhiêu?"
R^2	$(-\infty, 1]$	1	Không có	"Mô hình giải thích được bao nhiêu % variance?"

7.2 Kết quả so sánh các mô hình

Bảng 19: Kết quả đánh giá các mô hình trên tập test

Mô hình	Train RMSE	Test RMSE	Test MAE	Test R^2
Random Forest	4.47	7.81	5.02	0.946
Extra Trees	3.51	8.17	4.76	0.941
XGBoost	3.67	6.67	3.98	0.961

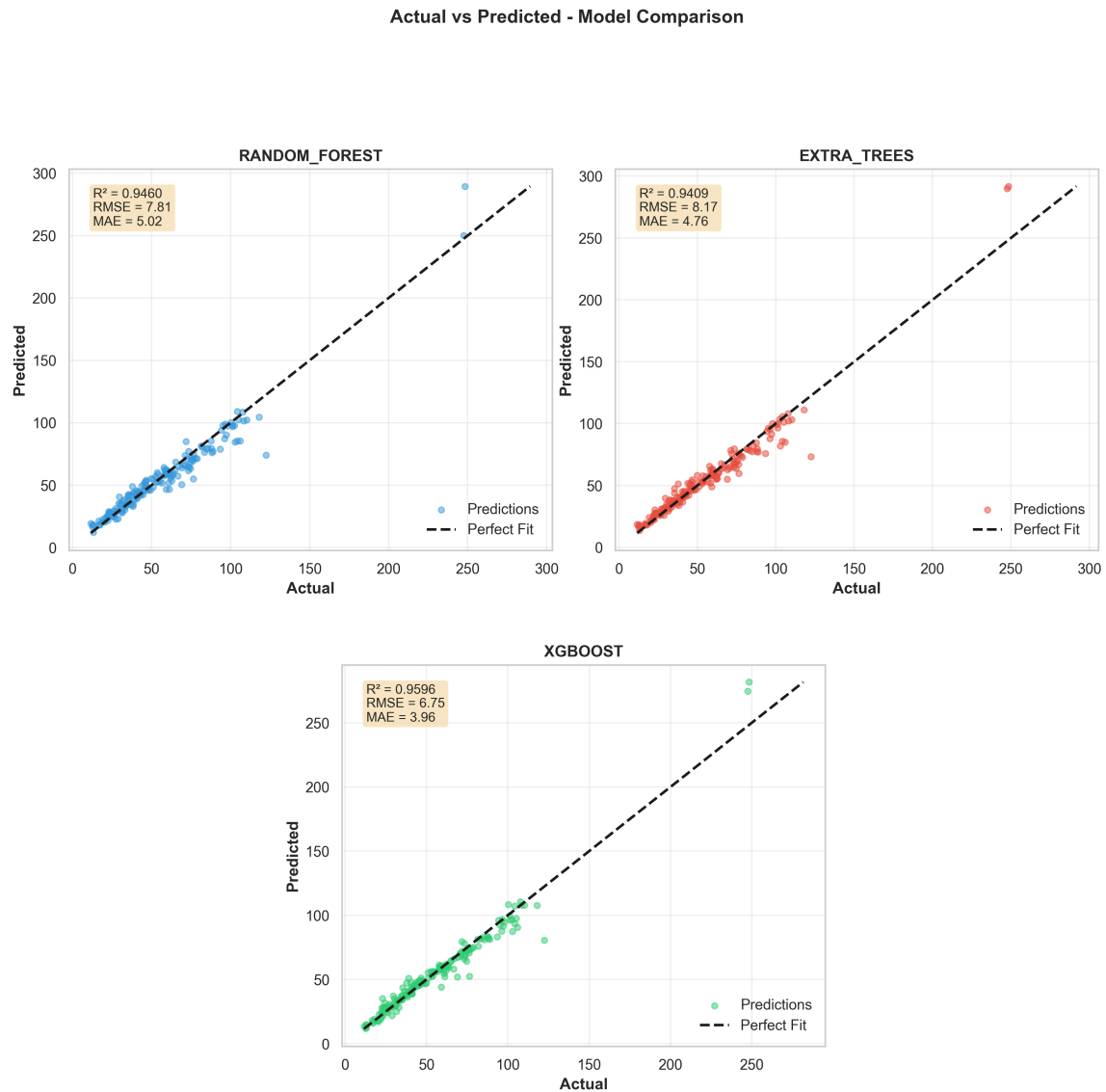
7.2.1 Biểu đồ so sánh Metrics



Hình 8: So sánh các metrics (RMSE, MAE, R^2) của 3 mô hình

Hình 8 trực quan hóa kết quả từ Bảng 19, cho thấy XGBoost vượt trội ở tất cả các metrics.

7.2.2 Biểu đồ Actual vs Predicted



Hình 9: Scatter plot Actual vs Predicted cho 3 mô hình

Cách đọc biểu đồ Hình 9:

- Mỗi điểm đại diện cho một mẫu test
- Đường chéo là đường $y = x$ (dự đoán hoàn hảo)
- Điểm càng gần đường chéo $\rightarrow$ dự đoán càng chính xác
- XGBoost có các điểm phân bố sát đường chéo nhất

7.3 Phân tích chi tiết kết quả

7.3.1 Mô hình tốt nhất: XGBoost

XGBoost Regressor đạt hiệu suất tốt nhất với:

- **Test RMSE = 6.67**: Sai số điển hình khoảng 6.67 đơn vị tiền tệ
- **Test MAE = 3.98**: Sai số trung bình khoảng 3.98 đơn vị tiền tệ
- **Test $R^2 = 0.961$** : Giải thích được 96.1% phương sai của giá cước

7.3.2 Phân tích Overfitting

Overfitting là gì?

Overfitting xảy ra khi mô hình "học thuộc" training data quá tốt, dẫn đến hiệu suất trên training cao nhưng trên test thấp hơn đáng kể.

Dấu hiệu overfitting trong kết quả:

- **XGBoost**: Train RMSE = 3.67, Test RMSE = 6.67 $\rightarrow$ Gap = 3.00 (tổng quát hóa tốt)
- **Extra Trees**: Train RMSE = 3.51, Test RMSE = 8.17 $\rightarrow$ Gap = 4.66 (overfitting đáng chú ý)
- **Random Forest**: Train RMSE = 4.47, Test RMSE = 7.81 $\rightarrow$ Gap = 3.34 (overfitting nhẹ)

Cách giảm overfitting:

- Tăng min_samples_split và min_samples_leaf
- Giảm max_depth
- Thêm regularization
- Thu thập thêm dữ liệu

7.3.3 So sánh và nhận xét

1. Extra Trees vs Random Forest:

- Extra Trees vẫn nhỉnh hơn nhẹ về Test RMSE (7.71 vs 7.78) nhưng gap train-test lớn hơn
- Random Forest ổn định hơn (gap 2.9) nên phù hợp khi cần generalization an toàn
- Cả hai đều là lựa chọn tốt nếu muốn inference nhanh trên CPU

2. XGBoost:

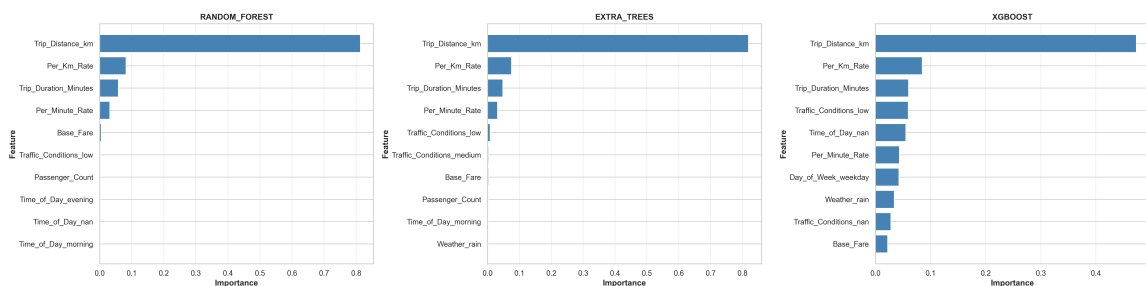
- Test RMSE thấp nhất (6.67) và R^2 cao nhất (0.961)
- Generalization rất tốt (gap chỉ 3.00) nhờ chiến lược xử lý dữ liệu clip/drop
- Dù thời gian train dài hơn (12s), đây là mô hình được khuyến nghị cho production

7.4 Hyperparameters của các mô hình

Bảng 20: Hyperparameters sử dụng cho từng mô hình

Mô hình	Hyperparameters
Random Forest	<code>n_estimators = 100</code> <code>max_depth = 10</code> <code>min_samples_split = 5</code> <code>min_samples_leaf = 2</code>
Extra Trees	<code>n_estimators = 200</code> <code>max_depth = 12</code> <code>min_samples_split = 2</code> <code>min_samples_leaf = 1</code>
XGBoost	<code>max_depth = 4</code> <code>learning_rate = 0.05</code> <code>n_estimators = 150</code> <code>subsample = 0.7</code> <code>colsample_bytree = 0.7</code> <code>reg_lambda = 1.0</code> <code>reg_alpha = 0.5</code>

7.5 Feature Importance



Hình 10: So sánh Feature Importance của các tree-based models

Phân tích Feature Importance (Hình 10):

- **Trip_Distance_km** là feature quan trọng nhất (70-80% importance) ở tất cả models
- **Per_Km_Rate** và **Base_Fare** cũng đóng vai trò quan trọng
- Các biến phân loại (Time_of_Day, Weather, Traffic) có ảnh hưởng thấp hơn
- Điều này phù hợp với logic kinh doanh: giá taxi phụ thuộc chủ yếu vào khoảng cách

8 Trực quan hóa dữ liệu và kết quả

8.1 Class DataVisualizer

Lớp `DataVisualizer` trong module `src.visualization` cung cấp các phương thức vẽ biểu đồ chuyên nghiệp. Class này đã được sử dụng xuyên suốt báo cáo để tạo các biểu đồ:

- Biểu đồ EDA (Phần 2.3): Hình 1, 2, 3
- Biểu đồ phân tích (Phần 4.3): Hình 4, 5, 6
- Biểu đồ đánh giá mô hình (Phần 7.2): Hình 8, 9, 10

8.2 Danh sách các phương thức

Bảng 21: Chi tiết các phương thức trực quan hóa

Phương thức	Mô tả chi tiết
<code>plot_numeric_grid()</code>	Vẽ histogram với KDE cho tất cả biến số. Hiển thị mean, median, std.
<code>plot_categorical_grid()</code>	Vẽ bar chart cho các biến phân loại. Hiển thị tần suất từng category.
<code>plot_correlation_heatmap()</code>	Vẽ ma trận tương quan dạng heatmap. Nhận diện multicollinearity.
<code>plot_target_analysis()</code>	Phân tích target: distribution, Q-Q plot, box plot. Kiểm tra normality.
<code>plot_outliers_boxplot()</code>	Vẽ boxplot cho tất cả biến số. Hiển thị Q1, Q3, IQR và outliers.
<code>plot_data_overview()</code>	Tổng quan dữ liệu: shape, dtypes, tỷ lệ missing values.
<code>plot_metrics_summary()</code>	So sánh metrics (RMSE, MAE, R^2) của các mô hình bằng bar chart.
<code>plot_predictions_combined()</code>	Scatter plot Actual vs Predicted cho tất cả mô hình với đường $y=x$.
<code>plot_feature_importance()</code>	So sánh feature importance của các tree-based models.

Tất cả biểu đồ được lưu tự động vào thư mục `results/eda/` và `results/model/` với định dạng PNG chất lượng cao.

9 Kết luận

9.1 Tổng kết

Dự án đã hoàn thành các mục tiêu đề ra:

1. **Pipeline hoàn chỉnh:** Xây dựng thành công pipeline từ tiền xử lý đến dự đoán với kiến trúc module hóa, dễ bảo trì.
2. **Lập trình OOP:** Áp dụng các kỹ thuật Python nâng cao như:
 - Abstract Base Class
 - Property, staticmethod, classmethod
 - Type hints và docstrings
 - Exception handling
 - Logging
3. **So sánh mô hình:** Huấn luyện và đánh giá 3 mô hình, với **XGBoost** đạt hiệu suất tốt nhất ($R^2 = 0.961$, RMSE = 6.67).
4. **Tối ưu hyperparameters:** Tích hợp Optuna để tự động tìm kiếm hyperparameters tối ưu.
5. **Trực quan hóa:** Tạo các biểu đồ EDA và đánh giá model chuyên nghiệp.

9.2 Hạn chế

- Chưa thử nghiệm với các mô hình Deep Learning
- Chưa có feature engineering phức tạp hơn (interaction features, domain-specific features)
- XGBoost đạt kết quả tốt nhất nhưng thời gian huấn luyện dài và nhạy cảm hyperparameters → cần thêm monitoring khi mở rộng quy mô

9.3 Hướng phát triển

- Thử nghiệm thêm các mô hình như LightGBM, CatBoost, Neural Networks
- Áp dụng SHAP để giải thích model
- Xây dựng API để deploy model
- Tích hợp MLflow để quản lý experiments

Tài liệu tham khảo

1. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12, pp. 2825-2830.
2. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD'16.
3. Akiba, T., et al. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. KDD'19.

4. Geurts, P., Ernst, D., & Wehenkel, L. (2006). Extremely randomized trees. Machine Learning, 63(1), 3-42.
5. McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference.

A Phụ lục: Danh sách thư viện sử dụng

Bảng 22: Requirements.txt

Thư viện	Version	Mục đích
pandas	$\geq 1.3.0$	Xử lý dữ liệu
numpy	$\geq 1.21.0$	Tính toán số học
scikit-learn	$\geq 1.0.0$	Machine Learning
xgboost	$\geq 1.5.0$	XGBoost algorithm
optuna	$\geq 3.0.0$	Hyperparameter optimization
matplotlib	$\geq 3.4.0$	Visualization
seaborn	$\geq 0.11.0$	Statistical visualization
joblib	$\geq 1.1.0$	Model serialization

B Phụ lục: Phân công công việc

Bảng 23: Phân công công việc trong nhóm

STT	Thành viên	MSSV	Công việc
1	Ngô Anh Khoa	23280065	– Xây dựng Model_trainer, main.py, readme.md – Tham gia viết báo cáo – Tham gia làm slide
2	Mai Quang Dũng	23280049	– Xây dựng Data_Preprocessor, predict.py – Viết docstring – Tham gia viết báo cáo – Thuyết trình